

Prinsip Desain, Atribut Kualitas, dan Trade-off

RICKARD ELSEN

Capaian pembelajaran

- **Asal RPS:** pertemuan 2–3, SubCPMK042, indikator 2.1–2.2 dan 3.1–3.2
- **Target:** menerapkan prinsip desain untuk menganalisis kualitas dan menulis skenario yang mendasari keputusan arsitektur
- **Persiapan mandiri – prinsip:** abstraction, separation of concerns, information hiding, coupling–cohesion, modularity
- **Persiapan mandiri – atribut:** maintainability, scalability, performance, availability, reliability, security, usability, testability
- **Hasil akhir:** matriks prinsip–dampak dan skenario kualitas untuk Tugas 1

Alur sesi

Bagian 1

Pemisahan tanggung jawab

Petakan persetujuan, penyimpanan, notifikasi; abstraksi dan information hiding

Bagian 2

Coupling–cohesion dan smells

Aturan tersebar vs pemilik aturan jelas; SOLID/GRASP

Bagian 3

Skenario kualitas

Enam bagian skenario; contoh persetujuan serentak

Bagian 4

Trade-off teknis

Konsistensi vs latensi, asinkron vs kegagalan, kebijakan vs abstraksi

Bagian 5

Bukti dan asumsi

Target teruji, asumsi, dan klaim tanpa ukuran

Setelah sesi: analisis batas modul dan tulis skenario kualitas → matriks prinsip–dampak untuk Tugas 1

Persetujuan, penyimpanan, dan notifikasi adalah tiga tanggung jawab terpisah



Satu perubahan kebijakan cukup diubah di satu modul

TANPA ABSTRAKSI

- Cek kuota “maks. 3 alat” ditulis di controller, layar persetujuan, dan job laporan
- Pemanggil membaca field nilaiAlat langsung untuk memutuskan
- **Dampak:** ubah dan uji ulang 3+ tempat; satu terlewat membuat aturan tidak konsisten

DENGAN INFORMATION HIDING

- **Abstraksi:** pemanggil hanya tahu bolehDisetujui(permintaan, penyetuju)
- **Disembunyikan:** angka kuota dan ambang nilai alat ada di dalam KebijakanPersetujuan
- **Dampak:** ubah satu kelas; pemanggil dan pengujiannya tetap

Perubahan kebijakan: kuota naik dari 3 menjadi 5 alat, dan alat bernilai tinggi wajib disetujui dosen

Pemilik aturan yang jelas menurunkan coupling

ATURAN TERSEBAR

- **Coupling tinggi:** empat modul bergantung pada detail status alat
- **Cohesion rendah:** controller memegang validasi, SQL, dan format email sekaligus
- **Gejala:** satu perubahan kecil memaksa edit di banyak file (shotgun surgery)

PEMILIK ATURAN JELAS

- **Coupling rendah:** modul lain bergantung pada antarmuka, bukan detail
- **Cohesion tinggi:** tiap kelas punya satu alasan untuk berubah
- **Testability:** aturan diuji terpisah dengan repositori dan notifikator tiruan

Cara mengukur: hitung jumlah modul yang harus diubah untuk satu perubahan aturan

SOLID dan GRASP memberi nama pada batas tanggung jawab yang baik

Prinsip	Pertanyaan desain	Contoh di sistem peminjaman
SRP (SOLID)	Apakah kelas punya satu alasan berubah?	Notifikator tidak tahu aturan kuota
OCP (SOLID)	Bisakah aturan baru ditambah tanpa mengubah kode lama?	Aturan alat bernilai tinggi menjadi kelas aturan baru
DIP (SOLID)	Apakah modul inti bergantung pada abstraksi?	Service memakai antarmuka repositori, bukan driver basis data
Information Expert (GRASP)	Siapa yang memegang data untuk memutuskan?	KebijakanPersetujuan memegang kuota dan ambang nilai
Controller (GRASP)	Siapa yang menerima event sistem?	PeminjamanService menerima “setujui permintaan”
Low Coupling, High Cohesion (GRASP)	Apakah dependensi minimal dan tugas fokus?	Tiga modul dengan dependensi satu arah

Kelas serba bisa dan dependensi siklik menandakan batas modul yang salah

Kelas serba bisa (god class)

Gejala

Satu kelas mengurus validasi, SQL, email, dan laporan

Dampak

Maintainability dan testability turun; tiap perubahan menyentuh kelas yang sama

Perbaikan

Pecah per alasan berubah (SRP); pindahkan aturan ke pemilik datanya (Information Expert)

Dependensi siklik

Gejala

Peminjaman memanggil Notifikasi, lalu Notifikasi memanggil balik Peminjaman untuk membaca status

Dampak

Modul tidak bisa diuji atau dirilis terpisah

Perbaikan

Balik arah dependensi lewat antarmuka (DIP), atau kirim event berisi data yang dibutuhkan

Skenario kualitas yang lengkap punya enam bagian

1

Sumber

Siapa atau apa yang memicu: pengguna, sistem lain, penyerang

2

Pemicu

Kejadian yang datang ke sistem

3

Kondisi

Keadaan sistem saat itu: normal, beban puncak, gangguan

4

Artefak

Bagian sistem yang terkena

5

Respons

Apa yang dilakukan sistem

6

Ukuran respons

Angka yang bisa diuji: waktu, persentase, jumlah

Tanpa ukuran respons, skenario hanya keinginan dan tidak bisa diuji

Dua staf menyetujui alat yang sama serentak: hanya satu yang boleh berhasil

Bagian	Isi skenario
Sumber	Dua staf laboratorium
Pemicu	Keduanya menekan “Setujui” untuk dua permintaan atas alat yang sama, berselisih kurang dari 1 detik
Kondisi	Operasi normal pada jam sibuk awal semester
Artefak	KebijakanPersetujuan dan RepositoriPeminjaman (data ketersediaan alat)
Respons	Satu persetujuan tersimpan; yang lain ditolak dengan pesan “alat sudah dipinjam”
Ukuran respons	0 persetujuan ganda dalam 1.000 uji serentak; pesan penolakan tampil ≤ 2 detik
Atribut terkait	Reliability (integritas data), lalu performance

Setiap atribut kualitas butuh ukuran yang bisa diuji

Maintainability

Perubahan kebijakan selesai ≤ 1 hari kerja dan menyentuh 1 modul

Scalability

500 permintaan/menit tanpa latensi naik di atas 2 detik

Performance

Persetujuan tersimpan ≤ 1 detik (p95)

Availability

Layanan aktif 99,5% selama jam operasional lab

Reliability

0 persetujuan ganda dalam 1.000 uji serentak

Security

Hanya peran penyetuju yang bisa menyetujui; semua upaya ditolak tercatat

Usability

Staf baru menyetujui permintaan pertama ≤ 3 menit tanpa panduan

Testability

Aturan persetujuan diuji tanpa basis data

Angka adalah contoh target untuk sistem peminjaman alat; tetapkan sendiri di Tugas 1

Setiap pilihan teknis membeli satu kualitas dengan biaya kualitas lain

Pilihan	Yang didapat	Yang dibayar	Pilih jika
Transaksi dengan penguncian data alat	Tidak ada persetujuan ganda (konsistensi)	Latensi naik saat permintaan antre di jam sibuk	Integritas data lebih penting daripada kecepatan
Notifikasi asinkron lewat antrean	Persetujuan tidak menunggu server email	Perlu retry, penanganan pesan ganda atau hilang, dan monitoring	Notifikasi boleh terlambat beberapa menit
Abstraksi untuk kebijakan persetujuan	Perubahan kebijakan cukup di satu modul	Lebih banyak antarmuka dan kelas untuk dipahami	Riwayat menunjukkan kebijakan berubah tiap semester

Klaim “cepat” atau “aman” baru menjadi bukti setelah punya ukuran dan hasil uji

Target teruji

Ukuran respons dan hasil uji tercatat

Contoh

Persetujuan tersimpan ≤ 1 detik (p95) pada uji beban 500 permintaan/menit

Tindakan

Pakai sebagai dasar keputusan arsitektur

Asumsi

Kondisi yang diyakini, belum diuji

Contoh

Puncak permintaan awal semester sekitar 500 per menit

Tindakan

Tulis eksplisit dan tentukan cara memvalidasinya

Klaim tanpa ukuran

Kata sifat tanpa angka atau kondisi

Contoh

“Sistem ini cepat dan aman”

Tindakan

Ubah menjadi skenario berukuran, atau hapus

Tugas mandiri setelah pemaparan

Aktivitas 1: analisis desain

- Analisis pseudo-code atau diagram dari dosen
- Usulkan batas tanggung jawab dan arah dependensi
- Tandai smells: kelas serba bisa, dependensi siklik

Aktivitas 2: skenario kualitas

- Tulis tiga skenario lengkap dengan ukuran respons
- Prioritaskan dua, lalu hubungkan ke smells dan pilihan perbaikan
- Bandingkan atribut yang tidak didemonstrasikan lewat bahan LMS

Hasil: matriks prinsip–dampak dan skenario kualitas untuk Tugas 1
Umpan balik dosen: satu batas modul dan satu ukuran kualitas

Format matriks prinsip–dampak untuk Tugas 1

Prinsip	Keputusan desain	Atribut terdampak	Ukuran dari skenario	Smell yang dicegah
Information hiding	Kuota dan ambang nilai disimpan di KebijakanPersetujuan	Naik: maintainability, testability	Perubahan kebijakan ≤ 1 hari, 1 modul	Shotgun surgery
Separation of concerns	Notifikasi asinkron dipisah dari persetujuan	Naik: performance. Biaya: reliability perlu retry	Persetujuan tersimpan ≤ 1 detik (p95)	Kelas serba bisa
Prinsip pilihan Anda	…	…	…	…

Tulis kualitas yang naik dan biaya yang ditanggung; tiap baris wajib punya ukuran.